

Instructions:

1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value "k", returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model. For each match, also list the corresponding distance/similarity score.

```
In [ ]: IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))

K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

```
In [ ]: # Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.

import os
from torchvision.datasets import Caltech101

dataset = Caltech101(root=os.path.abspath(""), download=True)
```

Files already downloaded and verified

```
In [ ]: # Find the k most similar images for a given Image ID for each feature model.

image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]

if image.mode != 'RGB':
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")
    image = image.convert('RGB')

print("Input image:\nImage ID: ", IMG_ID)
display(image)
print("Category:", category)
```

Input image:
Image ID: 0



Category: Faces

```
In [ ]: # Resize to 300x100 and partition to grid.  
  
image300x100 = image.resize((300, 100))  
  
from utils.utilities import partition_to_grid  
  
grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Color moments (CM10x10).  
  
# Find K most similar images by color moments.  
  
from feature_extractors.color_moments import get_color_vector  
  
color_vector = get_color_vector(grid)  
  
# Load color tensor database into memory.  
  
from utils.utilities import tensor_loader
```

```

color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(""))

from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.

# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)

for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])

```

Similar images by Color Moments using distance: Manhattan (cityblock)

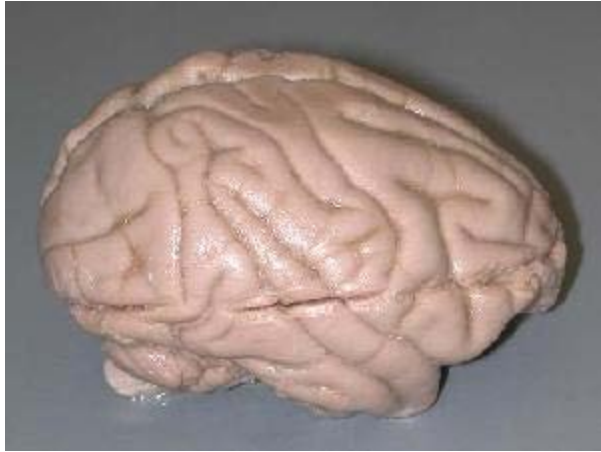
1. Image ID: 0 Distance: 0.0



2. Image ID: 435 Distance: 20543.707



3. Image ID: 3196 Distance: 20989.852



4. Image ID: 5160

Distance: 21095.695



5. Image ID: 5058

Distance: 21099.266



6. Image ID: 4 Distance: 21197.104



7. Image ID: 7615 Distance: 21968.61



8. Image ID: 275 Distance: 22091.668



9. Image ID: 4255 Distance: 22128.664



10. Image ID: 7258

Distance: 22196.605



```
In [ ]: # Convert to grayscale and partition to grid.  
  
from torchvision.transforms import Grayscale  
  
gs_image = Grayscale()(image300x100)
```



```
gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

```
In [ ]: # Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)

for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation

1. Image ID: 0 Distance: 0.0



2. Image ID: 7260

Distance: 0.1772551461243549



3. Image ID: 6384

Distance: 0.18155921487948135



4. Image ID: 7684

Distance: 0.1828890761653944



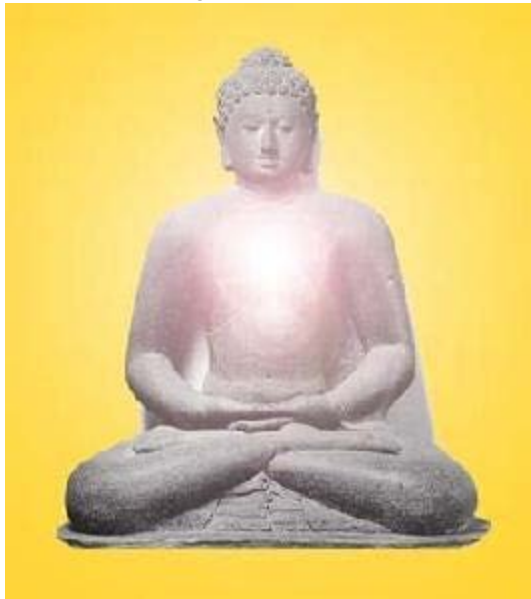
5. Image ID: 7621

Distance: 0.1832137524684716



6. Image ID: 3271

Distance: 0.18473419767448518



7. Image ID: 5655

Distance: 0.18711687502127605



8. Image ID: 5886

Distance: 0.18741789194374237



9. Image ID: 7000

Distance: 0.18751472764420007



10. Image ID: 5900

Distance: 0.18756641471349578



```
In [ ]: # Resize original image to 224x224 and convert to float tensor.
```

```
from torchvision.transforms import transforms
```

```
image224x224 = image.resize((224, 224))
```

```
img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

```
In [ ]: # ResNet pre-processing.
```

```
# Find K most similar images by each of the 3 layers chosen from ResNet-50.
```

```

from feature_extractors.resnet_50 import get_resnet50_feature_vectors

avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)

# Load each tensor database into memory

avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))

```

```

In [ ]: # ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)

for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])

```

```

Similar images by ResNet-AvgPool-1024 using distance: Cosine
1.      Image ID: 0           Distance: 0

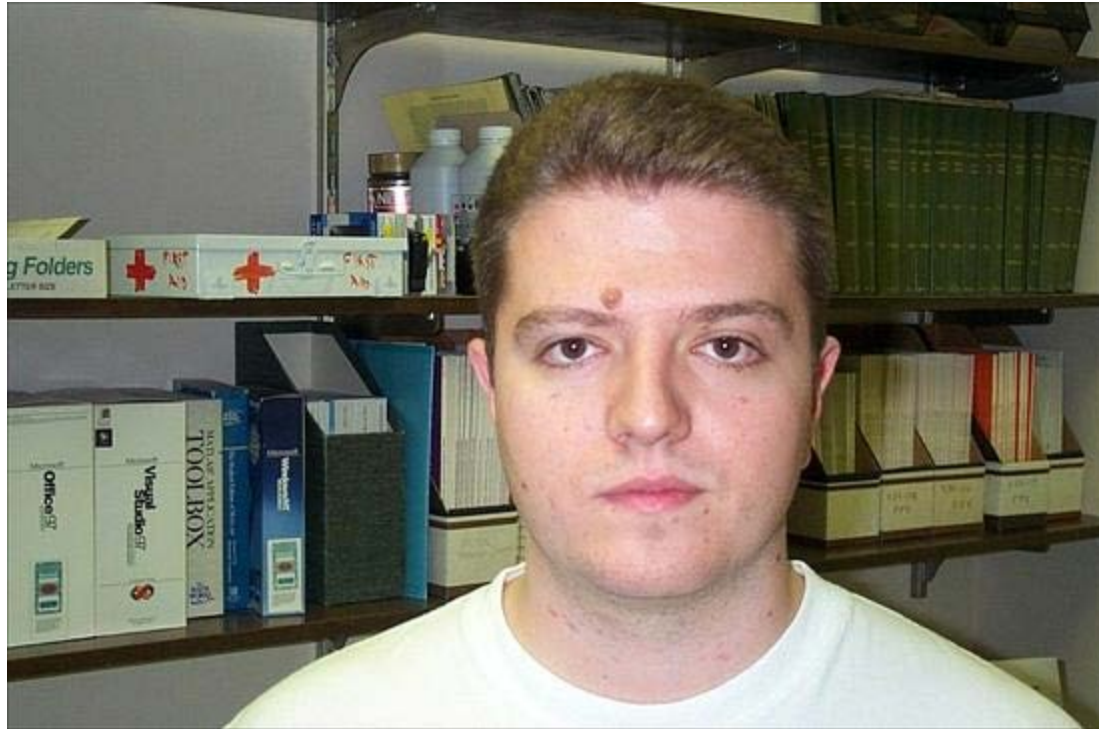
```



2. Image ID: 435 Distance: 0.14790743589401245



3. Image ID: 51 Distance: 0.18542933464050293



4. Image ID: 156 Distance: 0.19760167598724365



5. Image ID: 220 Distance: 0.20157980918884277



6. Image ID: 152 Distance: 0.2020755410194397



7. Image ID: 34 Distance: 0.2084168791770935



8. Image ID: 18 Distance: 0.213627099999084473



9. Image ID: 27 Distance: 0.2262974977493286



10. Image ID: 832 Distance: 0.22704273462295532



```
In [ ]: # ResNet-Layer3-1024.  
  
# Find K most similar images by ResNet-Layer3-1024.  
  
# Cosine similarity: We use Cosine since the output of the ResNet layer is  
# normalized, reducing the importance of magnitude. It is good at discerning  
# semantic or structural similarity rather than their absolute feature values.  
# It also works well with sparse feature spaces such as those found in CNNs.  
  
print("Similar images by ResNet-Layer3-1024 using distance: Cosine")  
  
top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)  
  
for i in range(len(top_k_by_layer3)):  
    id = top_k_by_layer3[i][1]  
    distance = top_k_by_layer3[i][0]  
  
    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)  
    display(dataset[id][0])
```

Similar images by ResNet-Layer3-1024 using distance: Cosine

1. Image ID: 0 Distance: 0



2. Image ID: 435 Distance: 0.02364814281463623



3. Image ID: 27 Distance: 0.028365373611450195



4. Image ID: 156 Distance: 0.02884727716445923



5. Image ID: 4 Distance: 0.03036510944366455



6. Image ID: 59 Distance: 0.0312839150428772



7. Image ID: 220 Distance: 0.03134727478027344



8. Image ID: 18 Distance: 0.03184032440185547



9. Image ID: 135 Distance: 0.03284412622451782



10. Image ID: 231 Distance: 0.0330316424369812



```
In [ ]: # ResNet-FC-1000.

# Find K most similar images by ResNet-FC-1000.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)

for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

Similar images by ResNet-FC-1000 using distance: Cosine

1. Image ID: 0 Distance: 0



2. Image ID: 435 Distance: 0.19422084093093872



3. Image ID: 18 Distance: 0.20374107360839844



4. Image ID: 156 Distance: 0.20885878801345825



5. Image ID: 220 Distance: 0.22211605310440063



6. Image ID: 49 Distance: 0.2309613823890686



7. Image ID: 152 Distance: 0.2348458170890808



8. Image ID: 51 Distance: 0.2357337474822998



9. Image ID: 469 Distance: 0.24090707302093506



10. Image ID: 55

Distance: 0.24337798357009888

